Lab report & Project

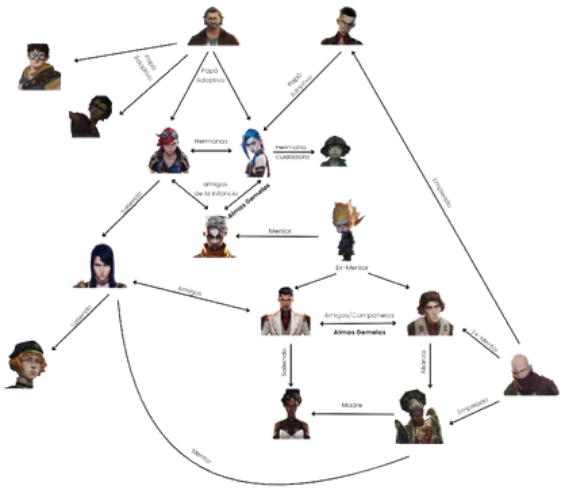
by Aurèle Habbard and Charles Didier

TD1: Repport

Knowledge Graph from the League of Legends Arcane Universe

Chosen Domain: League of Legends Universe

- Fictional but coherent universe
- Rich lore with places, characters, factions
- Well-suited for entity and relation extraction

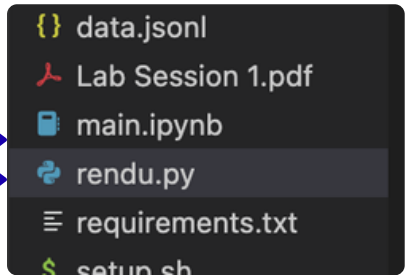


We chose the Arcane Universe as our domain because it provides a structured fictional world with consistent locations, characters, and organizations, making it ideal for building and analyzing a Knowledge Graph.

Data Source Selection

We collected around 15 URLs from the official League of Legends Wiki, mainly focused on regions (Piltover, Zaun, Noxus, Ixtal, etc.), champions, and factions, in order to understand the global structure and history of the universe.

```
urls = [
    "https://wiki.leagueoflegends.com/en-us/Universe:Runeterra",
    "https://wiki.leagueoflegends.com/en-us/Universe:Timeline",
    "https://wiki.leagueoflegends.com/en-us/Universe:Species",
    "https://wiki.leagueoflegends.com/en-us/Universe:Regions",
```



Code Organization Strategy

Separation of Concerns

→ main.ipynb: execution and visualization

→ rendu.py: core functions

All reusable functions were implemented in a dedicated file (rendu.py) and then imported into the main notebook. This separation makes the pipeline clearer and easier to follow.

Phase 1 — Text Extraction

Web Page Extraction

- HTML cleaned using trafilatura
- Main content only
- UI and layout noise removed

We used trafilatura to extract only the main textual content of each page, avoiding menus, footers, and interface elements.

raw page html

```
def extract_page(url, min_words=500):
    downloaded = trafilatura.fetch_url(url)

    text = trafilatura.extract(
    downloaded,
    include_comments=False,
    include_tables=False)
    if text is None:
        return None
    word_count = len(text.split())
    #if word_count < min_words:
    #    print(f"Attention l'url {url} ne contient que {word_count} mots")
    return {
    "url": url,
    "text": text,
    "word_count": word_count}
```

extraction pipe

{"url": "https://wiki.leagueoflegends.com/en-us/Universe:Runeterra", "text": "Universe:Runeterra\n-

data.jsonl

Phase 2 — Information Extraction with spaCy

Entity Extraction

Introduction to spaCy

- Industrial-grade NLP library
- Provides tokenization, parsing, NER

NLP Pipeline Loading

- Model: en_core_web_trf
- Transformer-based architecture

```python
import spacy
nlp = spacy.load("en_core_web_trf")

def extract_entities(text, allowed_labels={"PERSON", "ORG", "GPE", "EVENT", "DATE"}):
    doc = nlp(text)
    entities = []
    for ent in doc.ents:
        if ent.label_ in allowed_labels:
            if len(ent.text.strip()) > 2:
                entities.append({
                    "text": ent.text,
                    "label": ent.label_
                })
    return entities
```

- spaCy was used to process the cleaned text and extract meaningful linguistic information such as named entities and grammatical structure.
- We deliberately limited the number of entity labels to focus on domain-relevant concepts and reduce noise in the Knowledge Graph.

What is NER ?

NER = Named Entity Recognition

- PERSON: champions and key characters
- ORG: factions and institutions
- GPE: regions and cities
- EVENT and DATE: historical contex

```
Avarosa [PERSON]
Serylda [PERSON]
Watchers [ORG]
Lissandra [PERSON]
Watchers [ORG]
Fall of Icathia [PERSON]
Bai-Zek [GPE]
Shurima [GPE]
Ascended [PERSON]
centuries [DATE]
```

outputs

These are our results. At first glance, we can observe several duplicate entities. To address this issue, we added a deduplication step:

```
Number of entities: 123
```

```python
entities = list({(e["text"], e["label"]) for e in entities})
```

We managed to decrease the number of entities from **123 → 72**.

```
Number of entities: 72
```

Relation Extraction

Sentence-Level Analysis

- Text split into sentences
- Relations extracted per sentence
- Prevents cross-sentence noise

```python
def extract_relations(text):
    doc = nlp(text)
    relations = []
    for sent in doc.sents: # for each sentence :D
        ents = [ent for ent in sent.ents if ent.label_ in {"PERSON", "ORG", "GPE", "DATE"}]
        if len(ents) < 2:
            continue
        verbs = [token for token in sent if token.pos_ == "VERB"]
        if not verbs:
            continue
        verb = verbs[0].lemma_
        for i in range(len(ents) - 1):
            relations.append({
                "head": ents[i].text,
                "relation": verb,
                "tail": ents[i + 1].text,
                "sentence": sent.text
            })
    return relations
```

Relations were extracted sentence by sentence, as semantic relations rarely span across multiple sentences.

relation triple structure

```
(Entity A) → [verb] → (Entity B)
```

In the code, Entity A corresponds to the head, the relation to the verb, and Entity B to the tail.

Relation Definition

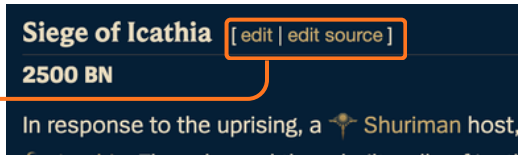
→ Verb lemma used as relation label
→ Entities appearing in the same sentence
→ Connected via a verb

Observed Issue: residual HTML content

Some extracted relations were incorrect, like “edit”, which originates from wiki interface elements.

```
Avarosa --[kill]--> Lissandra
Avarosa --[take]--> Serylda
Watchers --[cover]--> Lissandra
Lissandra --[cover]--> Watchers
Fall of Icathia --[edit]--> Bai-Zek
```

outputs



Major issue: Excessive and Irrelevant Text Content

- Extracted pages contained around 5,000 words
- But on the web page the plot is only ~**1,500 words**
- More than half of the text is out of scope..
- Non-narrative sections polluted the dataset

This is TOO MUCH

```
Processing url: 0  → Saved (4485 words)
Processing url: 1  → Saved (5002 words)
Processing url: 2  → Saved (5568 words)
Processing url: 3  → Saved (5964 words)
Processing url: 4  → Saved (5105 words)
Processing url: 5  → Saved (5273 words)
Processing url: 6  → Saved (6007 words)
Processing url: 7  → Saved (5171 words)
Processing url: 8  → Saved (4970 words)
```

Impact on number of entities & relations

```
Number of entities: 421
Sample entities:
Roger Craig Smith (PERSON)
Antoine Beaumier (PERSON)
Jerome Combe (PERSON)
Ophelie Senac (PERSON)
Renni (PERSON)
Diane Moscet (PERSON)
Council (ORG)
```

Input Text Quality Matters

We used BeautifulSoup to extract only the Plot section of each page, ignoring all other content. This reduced noise and improved entity and relation extraction.

```
<div class="mw-heading mw-heading2">
  <h2 id="Plot">Plot</h2>
  <span class="mw-editsection">…</span>
</div>
<p>
  "Young sisters "
  <span class="inline-image character-background-icon label-
  " and "
  <span class="inline-image character-background-icon label-
  " are seen amidst the aftermath of a devastating battle wh
  <span style="white-space:pre;">…</span>
  " and "
  <span style="white-space:pre;">…</span>
  ". The corpses of their parents lay among the fallen. "
  <span class="inline-image character-background-icon label-
  ", who had led the Zaun revolutionaries into battle, takes
</p>
<p>
```

Presence of Off-Domain Information

- Actor names frequently appeared in extracted text
- Production and casting details were included
- Netflix-related metadata polluted the content

```python
def extract_plot(url):
    html = trafilatura.fetch_url(url)
    if html is None:
        return None

    soup = BeautifulSoup(html, "html.parser")

    content = soup.find("div", class_="mw-parser-output")
    if content is None:
        return None

    plot_header = content.find("h2", id="Plot")
    if plot_header is None:
        return None

    paragraphs = []
    for elem in plot_header.find_all_next():
        if elem.name == "h2":
            break
        if elem.name == "p":
            paragraphs.append(elem.get_text(" ", strip=True))

    return "\n".join(paragraphs)
```

```
": 6007}
{"url": "https://wiki.leagueoflegends.com/en-us/Universe:A
sode_8", "text": "Oil and Water (Arcane: S01E08)\nTencent
h - audio description\nArabic\nBrazilian Portuguese\nChine
anish\nFinnish\nFrench\nGerman\nGreek\nHungarian\nHebrew\n
nKorean\nLatin Spanish\nMalay\nNorwegian\nPolish\nRomanian
ish\nUkranian\nVietnamese\nEnglish\nEnglish - audio descri
guese\nChinese\nCzech\nDanish\nEuropean Spanish\nFinnish\n
an\nHebrew\nIndonesian\nItalian\nJapanese\nKorean\nLatin S
sh\nRomanian\nRussian\nSwedish\nThai\nTurkish\nUkranian\n\
e eighth episode of the first season of the League of Lege
```

previous output

New output

```
(web_sem_env) (base) → TD1 python3 test.py
Young sisters Vi and Powder are seen amidst the after
took place upon the Bridge of Progress connecting Pil
ir parents lay among the fallen. Vander , who had led
le, takes pity on the two children and carries them t
Some years later, Vi leads Powder alongside their ado
a heist that begins upon the rooftops of Piltover. Po
 afraid to jump a gap between buildings but Vi assure
nearly falters.
```

The number of extracted entities (29) is now coherent and domain-relevant.

Entities correspond exclusively to actual characters from the series

```
Number of entities: 29
Number of relations: 105
Sample entities:
Jinx (PERSON)
Mel (PERSON)
Silco (ORG)
Sevika (PERSON)
Zaun (GPE)
Silco (PERSON)
Renni (PERSON)

Vi --[set]--> Caitlyn
Caitlyn --[set]--> Powder
Vi --[refuse]--> Jinx
Jinx --[refuse]--> Caitlyn
Vi --[promise]--> Jinx
Silco --[attempt]--> Jinx
Vi --[accuse]--> Jinx
Jinx --[hear]--> Mylo
```

Next, we aim to extract more relations and enrich them with family-based links (father, mother, siblings).

Mandatory question

Methodology — Navigation and Pipeline

We manually selected a small set of League of Legends Wiki pages and fetched them using trafilatura. Only the Plot section was extracted using BeautifulSoup

Named entities and relations were then extracted with spaCy and stored in a structured CSV

Ambiguity Analysis

Some PERSON entities initially referred to voice actors rather than story characters.

Generic terms like Council were sometimes ambiguous between fictional and real organizations.

Certain descriptive phrases were misclassified as EVENTS due to limited context.

Scaling Reflection

At larger scale, crawling would need to be automated with URL filtering and rate limiting.

Text extraction and NLP processing would require parallelization and batching.

Additional normalization and deduplication would be necessary to maintain graph quality.